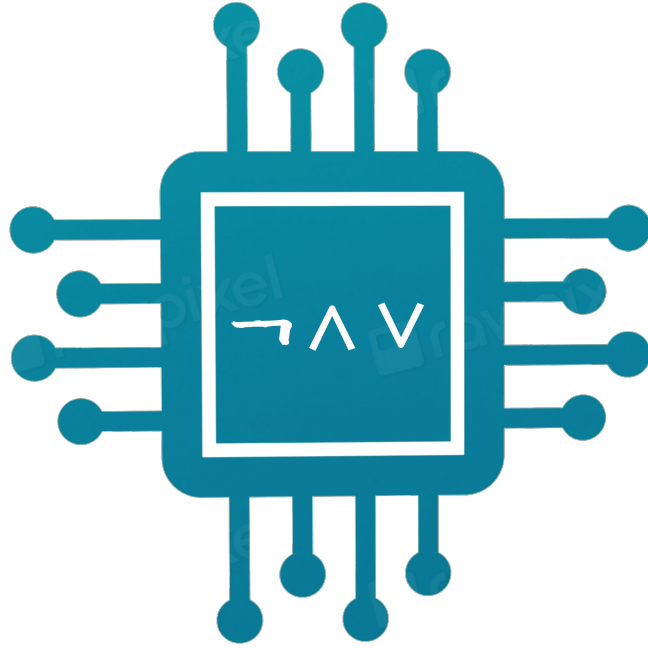




UNIVERSITY OF STUDY FLORENCE
COMPUTING ENGINEERING DEPARTMENT

Notandor

Unity Project

Authors:

Francesco Bonaiuti,
Tommaso Ciccotti,
Martina Schirone

Course:

Game Development [B032436]

Student Id:

7195553,
7196985,
7205227

Professor:

Tiezzi Francesco

Indice

1	Introduction	2
1.1	Game Overview	2
1.2	Challenges We Faced	2
1.3	Resources Used	2
2	Menu, Saving System and Option Settings	3
2.1	Main Menu	3
2.2	Option Settings	3
2.3	Pause Menu	4
2.4	Saving Menu and Saving System	4
2.5	GameState Script	4
3	Game System	5
3.1	Grid System	5
3.1.1	Scripts	5
3.1.2	Prefab	5
3.2	Interacting system	6
3.2.1	Layers	6
3.2.2	Scripts	6
3.3	Grabbing system	7
3.3.1	Layers	7
3.3.2	Scripts	7
3.4	Laser system	9
3.4.1	Layers	9
3.4.2	Scripts	9
3.4.3	Prefab	9
3.4.4	Colors logic	10
4	NPC Interactions And Cutscene Management	11
4.1	NPC Interactions	11
4.2	Cutscene Management	11
4.2.1	Main Cutscenes	11
4.2.2	Level Cutscenes	11
4.3	Scene Changer	12
4.4	Block Selection Indicator	12
5	Future Implementations	13
5.1	Mechanics	13
5.1.1	Rotation & Reflection	13
5.1.2	Enemy Mechanics	13
5.1.3	Height Levels	13
5.1.4	New Tiles	13
5.1.5	Grid System	13
5.2	Levels and Quality of Life	13
5.2.1	Additional Levels	13
5.2.2	Tile Visuals	13

1 Introduction

This report presents the "**Notandor**" project, a project developed in order to pass the *Game Development* exam of the Bachelor's Degree in Computer Engineering at the University of Florence.

This project was developed by Francesco Bonaiuti (SI 7195553), Tommaso Ciccotti (SI 7196985) and Martina Schirone (SI 7205227) between April and May 2026 and therefore referring to the academic year 2025/2026.

The source code of this entire project can be found under this link [Notandor Repository](#).

1.1 Game Overview

The game is simple in its core design. It consists of **four levels** in total, each taking place inside a single room. In every room, the only door leading to the next level is blocked by a **giant green spider**.

In front of the spider there is a **lamp**, while the room itself contains different types of blocks that replicate the behavior of logical operators such as **NOT**, **OR**, and **AND**.

The player can also interact with movable blocks and emitter blocks, which generate either green or red laser beams. Additionally, an NPC is present in the room to guide the player through the mechanics of the game, explaining the functionality of the different blocks and providing a tutorial experience.

The lore of the game is introduced through an initial cutscene. The rooms explored by the player are actually different sections of an infected computer system that has been overrun by bugs.

The giant spiders symbolize these bugs.

In each level, the player must **solve a puzzle** by correctly positioning and interacting with the blocks in order to activate the lamp.

Once the lamp is lit, the bug is resolved: the spider dies, the door opens, and the player can proceed to the next level. By progressing through the levels, the player gradually fixes the infected computer and restores its functionality.

1.2 Challenges We Faced

During the development of the game, we asked ourselves: "*What is the best way to create this game? What do we need to do?*"

After much thought, we came to the conclusion that the most important elements needed to create a playable game were the following challenges, each of which had to be solved individually by a member of the team:

**Menu, Saving System, and
Option Settings**

Game Systems

**NPC Interactions And
Cutscene Menagment**

1.3 Resources Used

The project was entirely developed using the **Unity Game Engine**.

Several free assets and Unity packages were also used during development to support both gameplay and presentation features.

- **Terminal User Interface Sound Effects Pack LITE Edition** – Used for keyboard clicking and interface sound effects.
- **Free Ambient & Lo-Fi Sci-Fi Music Starter Pack** – Used for the game's background music and ambient audio.
- **Jammo Character** – Used for the player character model.
- **Quarter View 3D Action Assets Pack** – Used for the NPC character model.
- **Sci-Fi Lab Kit** – Used for the environment, buildings, and puzzle block models.
- **Spider Polygon** – Used for the spider enemy model representing the "bugs".

2 Menu, Saving System and Option Settings

All menus in the game share the same basic structure. Each menu is built using a **Canvas** containing organized UI containers that include buttons, text, and interactive elements.

2.1 Main Menu

The Main Menu is the first interface shown when the player launches the game and the last one displayed before quitting. It is managed by a dedicated **MenuController** and **SavingController** script that allows the player to:

- **Start a new game**
- **Load or delete a previously saved game**
- **Change the options settings**
- **Quit the game**

When starting a new game, a confirmation popup is displayed before proceeding. Similarly, when deleting a save file, the player is warned through a **confirmation popup** that all progress associated with that slot will be permanently lost.

The load game section provides **three save slots**. Each slot can either be loaded or deleted through dedicated buttons. All saving functions are implemented via the **SavingController** script, later used for in game **Save Menu**.

All save files, settings and player preferences are managed with Unity's **PlayerPrefs** system.

The **MenuController** script is built around four main methods responsible for managing and updating the game's settings.

- **Load** – Loads all previously saved settings and player preferences stored in the PlayerPrefs.
- **Apply** – Saves the current settings selected by the player.
- **Set** – Updates individual settings values in real time based on the player's interactions with the UI elements.
- **Reset** – Restores all settings to their default values.

2.2 Option Settings

The Options settings are divided into **three different submenus**, each dedicated to a specific category.

- **Audio Settings** – Allows the player to adjust the master volume, music volume, and sound effects volume through sliders.
- **Graphics Settings** – Allows the player to modify brightness, graphics quality (low, medium, high), screen resolution, and fullscreen mode.
- **Gameplay Settings** – Allows the player to adjust mouse sensitivity, enable or disable the **Invert Y** option, view the game controls, and select the preferred language.

The game supports three different languages: English, Italian, and French.

The audio system is managed through a dedicated **Audio Mixer** that controls the overall audio output and dynamically adjust volume levels for music, sound effects, and master audio channels.

All graphics-related settings and visual quality options are handled using Unity's **Universal Render Pipeline (URP)**.

Methods Load, Set and Apply Examples

```
private void LoadVolume()
{
    musicVolumeSlider.value = PlayerPrefs.GetFloat("MusicVolume");
    mainVolumeSlider.value = PlayerPrefs.GetFloat("MainVolume");
    soundEffectsVolumeSlider.value = PlayerPrefs.GetFloat("SoundEffectsVolume");
}
```

Listing 1: **MenuController** Script: Load Audio Settings

```
public void SetMainVolume(float volume)
{
    mainVolumeValue.text = volume.ToString("0.0");

    // Convert the linear volume (0-100) to decibels for the audio mixer
    float normalized = volume / 100f;
    normalized = Mathf.Clamp(normalized, 0.0001f, 1f);
    float db = Mathf.Log10(normalized) * 20;

    audioMixer.SetFloat("master", db);
}
```

Listing 2: **MenuController** Script: Set Main Volume

```
public void VolumeApply()
{
    PlayButtonSoundEffect();
    PlayerPrefs.SetFloat("MainVolume", mainVolumeSlider.value);
    PlayerPrefs.SetFloat("MusicVolume", musicVolumeSlider.value);
    PlayerPrefs.SetFloat("SoundEffectsVolume", soundEffectsVolumeSlider.value);
}
```

Listing 3: **MenuController** Script: Apply and Save Audio Settings

2.3 Pause Menu

The Pause Menu contains all of the options setting functionalities available in the Main Menu thanks to the **MenuController** script. Additionally a dedicated **PauseMenuController** script introduces **two gameplay-specific functionalities**:

- **Resume** – Allows the player to continue the current game session.
- **Restart** – Reloads the current scene and restarts the level from the beginning.

2.4 Saving Menu and Saving System

The Saving Menu is managed by a dedicated **SavingController** script. It allows the player to save the current game progress into one of the available save slots. If the selected slot already contains saved data, an overwrite confirmation popup is displayed before replacing the previous save.

Once a slot is selected, the game stores information such as the **current date** and the active **scene name** using Unity's **PlayerPrefs** system. This information can later be accessed through the Load Menu in the Main Menu.

2.5 GameState Script

The **GameState** script is responsible for managing the different states of the game and preventing conflicting UI interactions. Its main purpose is to ensure that multiple menus **cannot be opened simultaneously**, avoiding interface overlap and unintended player input.

The script uses static global boolean variables to keep track of important game conditions, such as whether the game is currently paused or if the save menu is open.

```
public static class GameState
{
    public static bool IsPaused;
    public static bool IsSaveMenuOpen;
}
```

Listing 4: **GameState**: Simple State Management Script

3 Game System

The game is based on four principal subsystems. During the development of the systems multiple custom layers have been created and used.

Subsystems:

- Grid system
- Interacting system
- Grabbing systems
- Laser system

Layers:

- Interactable
- Movable
- Grabbed
- GrabbedInert
- Laser
- Tile
- Player

3.1 Grid System

The grid system controls the creation and interaction with the grid and defines how the different types of tiles works and how they are created.

3.1.1 Scripts

The scripts that allows this system to work are :

- **GridRuntime** - The main script responsible of controlling and creating the grid. It defines the dimensions, layout and the space between the tiles. It checks if the grid exist and if it doesn't it creates a new one, it's always executed. This script uses **if UNITY_EDITOR** to specify behaviors to use only in the editor and not during run time.
- **GridTileBase** - It's the abstract class that defines all the functionalities and attributes in common to all the tile types. All the tiles tell the player if he should continue to move and how his movement should change.
- **NormalTile** - It's the script of the normal tile, it doesn't alter the movement and tells the player to stop after the movement.
- **IceTile** - It controls the ice's behavior, it increases the movement speed and tells the player to continue to move.
- **BounceTile** - Continues the movement but inverts the direction..

3.1.2 Prefab

This system uses 4 objects:

- **Grid** - Concrete object that contains the grid in **GridVisual**.
- **Ice** - It's the prefab for ice tiles.
- **Point** - Normal tile prefab.
- **Bounce** - Bouncing tiles prefab.

Grid creation

```
void GenerateGrid()
{
    // Checks for GridVisual duplicates

    //Create grid visual
    gridParent = new GameObject("GridVisual");
    gridParent.transform.SetParent(transform);
    #if UNITY_EDITOR
        //Shows the scene as modified
    #endif
}
```

```

        Undo.RegisterCreatedObjectUndo(gridParent, "Create Grid");
        EditorUtility.SetDirty(gridParent);
    #endif
    for (int x = -halfWidth; x <= halfWidth; x++)
    {
        for (int z = -halfHeight; z <= halfHeight; z++)
        {
            int ix = x + halfWidth;
            int iz = z + halfHeight;
            int index = tileMap[ix, iz];

            //Check if the index is valid

            GameObject prefab = tileTypes[index].prefab;
            Vector3 pos = new Vector3(x * gridSize, 0.01f, z * gridSize);

            #if UNITY_EDITOR
                GameObject tile =
                    (GameObject)PrefabUtility.InstantiatePrefab(prefab);
                tile.transform.SetParent(gridParent.transform);
                tile.transform.position = pos;
                tile.transform.rotation = Quaternion.identity;
                Undo.RegisterCreatedObjectUndo(tile, "Create Tile");
                EditorUtility.SetDirty(tile);
            #else
                Instantiate(prefab, pos, Quaternion.identity,
                    gridParent.transform);
            #endif
        }
    }
}

```

Listing 5: GridRuntime Script: Generate grid

3.2 Interacting system

This system controls how the player interacts with the interactable objects. It defines the distance of interaction and what happens during it. It's also responsible for deciding which layers are considered interactable by the user.

3.2.1 Layers

This system uses the layer **Interactable**.

3.2.2 Scripts

This system presents two scripts:

- **Interactable** - This is the interface for all the interactable objects, it defines all the needed methods for interacting and doing things when the player enters or leaves the range. It controls the dialogue prompt and its window.
- **PlayerInteract** - This script allow the player to detect and interact with interactable objects up to a maximum distance and a certain angle from him. Moreover it controls the input and conditions needed to start the interaction.

Player interact

```

void DetectInteractable()
{
    Vector3 origin = transform.TransformPoint(controller.center);
    Collider[] hits = Physics.OverlapSphere(
        origin,
        interactDistance,
        interactMask
    );

    // Attributes to keep track of the best object to interact with

    foreach (Collider col in hits)
    {
        // Obtaining the Interactable component from the collider and checking if
        it's interactable
    }
}

```

```

        Interactable interactable = col.GetComponent<Interactable>();
        //Finiding the closest point of the object and computing the scalar product of
        the direction

        // Update the status if the scalar product is better
        if (dot > bestDot)
        {
            bestDot = dot;
            bestInteractable = interactable;
        }
    }
    if (bestInteractable != currentInteractable)
    {
        // If the object has changed it calls OnExitRange
        currentInteractable?.OnExitRange();
        currentInteractable = bestInteractable;
        currentInteractable?.OnEnterRange();
    }
    // If the object isn't interactable anymore it resets the status
    if (currentInteractable != null &&
        !currentInteractable.canInteract)
    {
        currentInteractable.OnExitRange();
        currentInteractable = null;
    }
}

```

Listing 6: **PlayerInteract Script**: Set Main Volume

3.3 Grabbing system

This system depends on the **Interacting system** to interact with the movable objects and on the **Grid system** to move the objects on the grid. It allows the player to grab, move and rotate the objects that allow such actions. Thanks to this the player can solve the puzzles in each level. Without the previous systems it cannot work.

3.3.1 Layers

The grabbed object it's given a new layer depending on the type of object while the original layer is saved, this is done so that the grabbed object and the player don't interfere with each other.

- **Grabbed** - This is given to objects belonging to the **Laser system** so that they can see each other:
- **GrabbedInert** - Given to the objects that don't belong to the **Laser system** so that they are ignored by the lasers but they still don't interfere with the player

3.3.2 Scripts

There are the two scripts that allows this system to work:

- **MovableObject** - It must be given to each game object which we want the player to be able to rotate or move. It has various options to allow for different possible actions. It extends **Interactable**. If the **Grid system** is not found in the scene it disables itself.
- **PlayerGrab** - This script is the hearth of this system. It controls the grabbing and movement of the objects, it checks if there are collisions and what type of tile the player is walking on and into. It also controls the interaction between the player and the objects it wants to grab. If the object would exit the grid or collide with something it stops the movement, moreover when the player grabs an object it makes sure the player remains close and looks at it. The movement and the player animation starts only after the whole movement path has been calculated. It checks if the **Grid** and **Interact system** are in the scene otherwise it disables himself.


```

void HandleMovement(Vector3 dir)
{
    // Checks if the object is not null or if it's already moving or rotating

    //Obtains initial object position

    List<Vector3> blockPath = new List<Vector3>();
    blockPath.Add(currentBlock);
    // Limits maximum number of steps for single input
    for (int i = 0; i < maxSteps; i++)
    {
        GridTileBase currentTile = GetTileAt(currentBlock);
        //Checks if the tile exists

        //The tile modifies the movement
        Vector3 nextBlock = currentTile.ModifyMovement(
            currentBlock,
            ref dir,
            grid.gridSize,
            obstacleMask
        );
        // Checks if the movement is not null

        //Player follows the object
        Vector3 nextPlayer = nextBlock - grabDirection * grid.tileDistance;
        // Checks if the new position is out of the grid

        // Checks the object collisions
        Collider blockCol = grabbedObject.GetComponent<Collider>();
        Vector3 blockCenter = nextBlock + blockCol.bounds.center -
            grabbedObject.transform.position;
        Vector3 blockHalfExtents = blockCol.bounds.extents * 0.9f;
        Collider[] blockHits = Physics.OverlapBox(
            blockCenter,
            blockHalfExtents,
            Quaternion.identity,
            obstacleMask
        );
        //If there are collisions with other objects it stops the movement

        // Player collisions
        float halfHeight = controller.height / 2f;
        Vector3 checkCenter = nextPlayer + Vector3.up * halfHeight;
        Vector3 checkHalfExtents = new Vector3(
            controller.radius,
            halfHeight,
            controller.radius
        ) * 0.8f;
        Collider[] hits = Physics.OverlapBox(
            checkCenter,
            checkHalfExtents,
            Quaternion.identity,
            obstacleMask
        );
        //If there are collisions with other objects it stops the movement

        currentBlock = nextBlock;
        // Add the new position to the path
        blockPath.Add(currentBlock);

        GridTileBase destTile = GetTileAt(currentBlock);
        // If the destination tile is null or it doesn't continue the movement it
        stops
    }
    // If the path contains more than one position
    if (blockPath.Count > 1)
    {
        isMoving = true;
        StartCoroutine(MoveStepByStep(blockPath, playerPath));
    }
    else
    {
        Debug.Log("<color=grey>Nessun movimento</color>");
    }
}

```

Listing 7: PlayerGrab Script: handle movement

3.4 Laser system

The laser system is what fuels the puzzles and allows to combine different logic ports and other objects, it's the core of the game together with the grabbing system..The logic ports evaluate their logic based on the color and numbers of the input lasers.

3.4.1 Layers

The custom layer used by this system is **Laser**.

3.4.2 Scripts

The scripts:

- **ILaserReceiver** - Base interface that defines how to react to an incoming ray with an abstract method.
- **LaserNode** - It centralizes the laser firing and drawing logic, the output direction evaluation and defines with which objects the laser interacts.
- **Reflector** - This class defines how the various objects belonging to this systems reflect and propagate every laser and what they must do after being hit. It controls the output direction of the laser or if the object must propagate the laser in the original direction. It extends **LaserNode** and implements **ILaserReceiver**.
- **logicGate** - It's the abstract class that's responsible for the logic and behavior of the logic ports. It extends **Reflector**. It defines how the ports reacts to different laser colors.
- **LogicOr** - Extends **LogicGate** and implements the Or evaluation logic.
- **LogicAnd** - Like the Or but implements the And logic.
- **LogicNot** - When active inverts the input color.
- **LaserInteractable** - This class implements **Interactable** and allow the player to activate the laser emitters.
- **LaserEmitter** - It extends **LaserNode** and controls the color,duration, direction and maximum distance of the emitted laser.
- **LaserSensor** - This script defines how the sensor works. It controls which laser color activates the sensor and what happens when the right laser hits it. It uses unity events to invoke other objects when it's activated or deactivated. It extends **Reflector**.

3.4.3 Prefab

The scripts are used by various concrete objects:

- **Or** - Or logic port
- **Not** - Not logic port
- **And** - And logic port.
- **Reflector** - Generic reflecting block
- **Emitter** - Block that originates the laser both in preview or active mode.
- **Sensor** - Block that receives the laser and evaluates what to do.

```
protected void FireRay(Vector3 origin, Vector3 direction, LaserColor color)
{
    Illuminate(color != LaserColor.Preview, color);
    //The raycast starts at the center of the object plus the offset
    Vector3 physicsOrigin = origin;
    Vector3 endPoint = physicsOrigin + direction * maxDistance;
    // Combined layermask with the obstacles and laser components
    LayerMask combinedMask = laserMask | blockMask;
    RaycastHit hit;
```

```

if (Physics.Raycast(physicsOrigin, direction, out hit, maxDistance,
    combinedMask))
{
    // If it hits update the laser end point
    endPoint = hit.point;
    int layer = hit.collider.gameObject.layer;
    // Checks if the object that has been hit is a laser object
    if (((1 << layer) & laserMask) != 0)
    {
        ILaserReceiver receiver = hit.collider.GetComponent<ILaserReceiver>();
        if (receiver != null)
        {
            receiver.OnLaserHit(hit.point, color);
        }
    }
}
// Drawing the laser
DrawLaser(origin, endPoint, color);
}

```

Listing 8: LaserNode Script: Fire ray

3.4.4 Colors logic

The logic ports evaluate the logic differently based on the laser's type. There are 3 types of laser:

- **Preview** - It defines a gray color that doesn't activate the logic evaluation and it's seen as a preview of the laser path.
- **Green** - It's seen as the logic True and as an active signal, it activates the evaluation logic.
- **Red** - It's the logic False and like the green it is seen as an active signal.

The ports are active only if they receive the required number of active inputs, until that moment if hit by a laser the output is the preview.

A	NOT
GREEN	RED
RED	GREEN
GRAY	GRAY

Figura 1: NOT Truth Table

A	B	AND
GREEN	GREEN	GREEN
GREEN	RED	RED
RED	GREEN	RED
RED	RED	RED
RED	GRAY	GRAY
GREEN	GRAY	GRAY
GRAY	RED	GRAY
GRAY	GREEN	GRAY
GRAY	GRAY	GRAY

Figura 2: AND Truth Table

A	B	OR
GREEN	GREEN	GREEN
GREEN	RED	GREEN
RED	GREEN	GREEN
RED	RED	RED
RED	GRAY	GRAY
GREEN	GRAY	GRAY
GRAY	RED	GRAY
GRAY	GREEN	GRAY
GRAY	GRAY	GRAY

Figura 3: OR Truth Table

4 NPC Interactions And Cutscene Management

In the following section we will examine how NPCs and cutscenes have been implemented, along with a few minor mechanics related to them, such as language switching and scene transitions. The main challenge faced in this part was **synchronization**, that is, ensuring that the system responded correctly to the player's actions by blocking and re-enabling timelines, canvases, and controls based on what was happening on screen.

4.1 NPC Interactions

NPCInteractable is a subclass of **Interactable** whose main difference is the ability to reference an object known as **Dialogue Data** and display it on screen via a **Canvas**. In order to translate the dialogue into different languages, **Dialogue Data** retrieves the text from a **Localization Table**, so that the language changes automatically whenever the player updates it from the settings menu. To make the experience more visually appealing, the text is not displayed all at once, but is instead printed character by character through a simple **for** loop inside the script.

Additionally, to prevent the player from moving during a dialogue, **NPCInteractable** calls **MovementInput** to disable movement until the conversation ends.

```
protected override void Interact()
{
    if (!dialogueActive)
    {
        dialogueActive = true;
        playerInteract?.SetInteracting(true);
        movementInput?.SetMovementEnabled(false); // blocca il movimento
                                                    durante il dialogo
        DialogueManager.Instance.StartDialogue(dialogue, OnDialogueEnd);
    }
    else
    {
        DialogueManager.Instance.NextLine();
    }
}
```

Listing 9: **NPCInteractable** Script: NPC interaction

4.2 Cutscene Management

Unity already includes a built-in system for managing cutscenes through the so-called **Timelines**, which allow animating and controlling virtually anything during a sequence. However, care must be taken regarding how this external mechanic interacts with the game scene and the player. For this reason, cutscenes are divided into two distinct situations.

4.2.1 Main Cutscenes

These are the two main cutscenes of the game. They are placed in dedicated scenes, separate from the rest of the game: one at the beginning and one at the end. They are fairly simple, making use of the basic animations provided by the assets found online, and tell a brief story: the protagonist's computer, which represents his home, has been infected by Bugs, and he must now travel through the levels to eliminate them.

The first cutscene is loaded when the player presses *New Game* from the main menu, while the second plays at the end of the 4th level.

Through a short script, **CutsceneSceneLoader**, these two scenes then transition to their respective next scenes as soon as the timeline has finished executing.

4.2.2 Level Cutscenes

Each level contains two cutscenes. The first is triggered when the green laser hits the lamp. The **LaserSensor** script calls **CutsceneTrigger**, which starts the cutscene. It deactivates the player's camera to hand control over to the **VirtualCamera**, restoring it once the timeline ends. At the end of the timeline, the script also deactivates the object containing the cutscene, preventing it from creating conflicts with the player's camera or other cutscenes.

As with the NPC interactions, to prevent the player from moving during the end-of-level cutscene, **CutsceneTrigger** calls **MovementInput** to disable movement until the sequence is complete.

The second cutscene is triggered when the player approaches the door. An invisible object with a **trigger collider** activates the associated script **DoorCutsceneLoader**, which starts the timeline. At the end of the timeline, a **fade** of approximately one second plays before transitioning to the next level, where the script **SceneStartFadeIn** removes the fade effect, allowing the new level to begin.

```
private IEnumerator PlayCutscene()
{
    // Attiva il GameObject della cutscene prima di avviarla
    if (timeline != null)
    {
        timeline.gameObject.SetActive(true); // <-- aggiunto
        timeline.Play();
    }

    yield return new WaitForSeconds((float)timeline.duration);

    yield return StartCoroutine(fader.FadeOut(fadeDuration));

    SceneManager.LoadScene(nextSceneName);
}
```

Listing 10: **DoorCutsceneLoader Script**: next level transition

4.3 Scene Changer

As already mentioned, scene transitions are handled by three different scripts, for three different situations:

- In the **main menu**, the scene is changed via the menu's own dedicated script.
- In the **main cutscenes**, the transition is handled by **CutsceneSceneLoader**, which waits for the timeline to finish.
- In the **levels**, the transition is managed by **DoorCutsceneLoader**, which waits for the player to approach the door before launching the timeline and the fade effect, and by **SceneStartFadeIn**, which removes the fade carried over from the previous level.

4.4 Block Selection Indicator

In each level there is a small red sphere, controlled by a script called **BlockSelectionIndicator**, which communicates with **PlayerInteract** to determine which block the player is currently looking at. When **PlayerInteract** detects a block, it notifies **BlockSelectionIndicator**, which in turn moves the red sphere above that block, signaling to the player which block they are looking at.

```
void Update()
{
    if (indicator == null || playerInteract == null) return;

    // Read the currently selected block using the public getter
    Interactable current = playerInteract.GetCurrentInteractable();

    Vector3 targetPos;

    if (current != null)
    {
        // No block selected: send the ball away
        targetPos = current.transform.position + Vector3.up * heightOffset;
    }
    else
    {
        // Nessun blocco selezionato: mandiamo la sfera lontano
        targetPos = hiddenPosition;
    }

    // Fluid movement (lerp), or instantaneous if followSpeed <= 0
    if (followSpeed > 0f)
        indicator.position = Vector3.Lerp(indicator.position, targetPos,
            Time.deltaTime * followSpeed);
    else
        indicator.position = targetPos;
}
```

Listing 11: **BlockSelectionIndicator Script**: move the sphere

5 Future Implementations

In this section, we'll look at the implementation of ideas we didn't have time to complete, or that we completed but didn't make it into the game's playable demo.

5.1 Mechanics

5.1.1 Rotation & Reflection

Reflection As previously explained, the direction of the laser beam for each block is determined by the **Reflector** script. Within the game, this script serves as the base class for all block types. However, it can also be used for other purposes. For instance, we created a block that reflects the laser beam in a different direction, functioning like a mirror, and a **LogicNot** variant that, in addition to altering the beam's color, also changes its direction. These blocks have already been partially implemented in the game, but did not find a place in any level due to time constraints.

Rotation Similarly, the game already includes a **Rotation** script, which allows the player to rotate the facing direction of blocks. Combined with the **Reflector** script, it enables the player to rotate blocks and alter the direction in which they reflect the laser beam. This mechanic is already present in the game and can be enabled via a toggle in the Unity Editor, but it was not used in the final levels because of time limitations. A brief dedicated level, referred to internally as **Level X**, was also created: after retrieving an item from a crate, the player would unlock the ability to rotate blocks and use it to complete the level, functioning as a sort of power-up introduction. This level is still present in the Editor, albeit with some minor issues.

5.1.2 Enemy Mechanics

We initially considered allowing the player to fight the Bugs as enemies in order to obtain the lamp to then activate through the puzzle. However, this would have simply replicated what we had already seen during lectures, and we did not consider it an interesting enough programming challenge.

5.1.3 Height Levels

We considered the possibility of adding blocks placed at different heights, for instance by allowing blocks to fall from upper platforms to lower ones, or to be pushed back up via special tiles. While interesting as a way to add depth to the game, it ultimately represented a minor challenge along the same lines as what we had already programmed, and we therefore decided to focus on other aspects.

5.1.4 New Tiles

Given how the game is currently programmed, adding new types of tiles, such as a tile that moves the player two steps at a time or one that damages the player, would be straightforward to implement. New tile types could therefore be added whenever needed. This would be easy to do as the system is modular so creating a new script extending `GridTileBase` and a new prefab would be enough.

5.1.5 Grid System

Moreover the grid system like now creates a default grid that can be personalized in the code or later manually in the editor tile by tile, an improvement would be to allow to customize the default grid in the editor through some parameters.

5.2 Levels and Quality of Life

5.2.1 Additional Levels

Additional levels could have been added, but this would have only meant reusing the blocks we had already created. We therefore did not consider it an interesting challenge.

5.2.2 Tile Visuals

We would have liked to use different tiles to represent the icy or bouncing floor. Unfortunately, lacking 3D modelling skills, we limited ourselves to combining assets found online or recolouring existing ones, as we did for instance with the AND and OR logic gate blocks.